

Raport końcowy projektu

Bot czy człowiek? Behawioralna biometria ruchu myszy
dla ochrony aplikacji webowej

Piotr Kosiara
Emilia Anczarska

17 maja 2026

Spis treści

1	Informacje organizacyjne	4
1.1	Skład zespołu	4
1.2	Podział prac	4
2	Wprowadzenie i motywacja	5
2.1	Kontekst problemu	5
2.2	Dlaczego behawioralna biometria myszy	5
2.3	Cel projektu	5
2.4	Zakres i świadome uproszczenia MVP	5
3	Architektura systemu	6
3.1	Widok komponentów	6
3.2	Przepływ danych end-to-end	6
3.3	Diagram przypadków użycia	7
3.4	Model danych (ERD)	8
4	Dokumentacja techniczna codebase	8
4.1	Struktura repozytorium	8
4.2	Frontend testbed (Vite + React + TypeScript)	9
4.2.1	Rola frontendu	9
4.2.2	Wygląd i funkcje interfejsu	9
4.2.3	Zbierane eventy i batching	10
4.2.4	Fingerprint klienta	11
4.3	Pozyskanie danych treningowych (human vs bot)	11
4.3.1	Jak zbierano dane klasy human	11
4.3.2	Jak zbierano dane klas botowych	11

4.3.3	Udział scenariuszy botowych	11
4.3.4	Charakterystyka botów i zamysł projektowy	11
4.3.5	Scenariusze eksperymentalne	12
4.4	Generacja i przygotowanie danych	12
4.4.1	Opis zebranych danych	12
4.4.2	Przygotowanie danych	12
4.4.3	Analiza jakości danych	13
4.5	Backend Collector API (FastAPI)	13
4.5.1	Warstwy backendu	13
4.5.2	Endpointy REST	13
4.6	Storage: PostgreSQL	14
4.7	Słownik zmiennych bazodanowych	14
4.8	Feature engineering	16
4.8.1	Przykładowe cechy dynamiczne	16
4.8.2	Cechy środowiskowe	16
4.8.3	Słownik zmiennych cechowych	16
4.9	Model i inferencja	18
4.9.1	Trening offline	18
4.9.2	Predykcja online	18
4.10	Policy engine / enforcement	18
4.11	DevOps i uruchomienie	19
4.12	Jakość i testy	19
5	Model finalny i raport wyników	19
5.1	Cel i zakres analizy modelu	19
5.2	Konfiguracja modelu finalnego	19
5.3	Uzasadnienie wyboru modelu	20
5.4	Jakie dane weszły do modelu	20
5.4.1	Poziom surowy	20
5.4.2	Poziom cech	20
5.5	Dobór zmiennych: proces i uzasadnienie	20
5.5.1	Przestrzeń kandydacka i finalny wybór	20
5.5.2	Kryteria doboru zmiennych	21
5.5.3	Co oznacza ten wybór w praktyce	21
5.6	Podział train/test i walidacja	21
5.6.1	Strategia podziału	21
5.6.2	Co oznaczają liczby w praktyce	21
5.6.3	Typ walidacji	21
5.7	Metryki ilościowe	22
5.7.1	Metryki z treningu (hold-out, GroupShuffleSplit)	22

5.7.2	Metryki ewaluacyjne (aktywny model)	22
5.8	Wizualizacje modelu	23
5.9	Interpretacja: co model rzeczywiście robi	24
5.9.1	Struktura decyzyjna drzewa: interpretacja warstw	24
5.9.2	Jak czytać diagram ważności cech	24
5.9.3	Znaczenie operacyjne (co to oznacza dla detekcji)	25
5.10	Reguły klasyfikacji (fragment)	25
5.11	Czy ewaluacja jest spójna z rzeczywistością?	25
5.11.1	Co jest spójne	25
5.11.2	Gdzie jest ryzyko przeszacowania	25
5.11.3	Najważniejsze ograniczenie obecnego raportu	25
5.12	Wpływ jakości danych na wyniki	26
5.13	Ograniczenia trafności wyników	26
5.14	Wniosek końcowy	26
6	Podsumowanie projektu	27
6.1	Co zostało osiągnięte	27
6.2	Wnioski praktyczne	27
6.3	Rekomendowane kolejne kroki	27
7	Materiały dodatkowe	27
7.1	Repozytorium projektu	27
7.2	Struktura datasetu	28
7.3	Rozmiar i skład danych końcowych	28
7.4	Odtwarzalność eksperymentu	28

1 Informacje organizacyjne

Pole	Informacja
Nazwa przedmiotu	ADAC – Analiza danych w cyberbezpieczeństwie
Semestr	26L
Nazwa projektu	Bot czy człowiek? Behawioralna biometria ruchu myszy dla ochrony aplikacji webowej
Nazwa zespołu	CapsLock
Numer zespołu	7
Repozytorium projektu	https://github.com/PiotrKosiara/Projekt_ADAC

Tabela 1: Podstawowe informacje organizacyjne projektu

1.1 Skład zespołu

Projekt został wykonany przez dwuosobowy zespół studentów kierunku Telekomunikacja, specjalność Teleinformatyka i Cyberbezpieczeństwo.

Członek zespołu	Kierunek i specjalność
Piotr Kosiara	Telekomunikacja, specjalność Teleinformatyka i Cyberbezpieczeństwo
Emilia Anczarska	Telekomunikacja, specjalność Teleinformatyka i Cyberbezpieczeństwo

Tabela 2: Członkowie zespołu projektowego

1.2 Podział prac

Tabela 3 przedstawia przyjęty podział zadań projektowych. Udziały procentowe sumują się do 100%.

Osoba	Zakres zadań	Udział
Piotr Kosiara	architektura backendu i API, pipeline feature engineering, trening i ewaluacja modelu, rozwój bot runnera, generacja danych botowych, Docker Compose, integracja pipeline’u end-to-end	50%
Emilia Anczarska	analiza problemu i wymagań, projekt scenariuszy testbedu, wsparcie zbierania danych human, opis działania frontendu, dokumentacja, interpretacja wyników, przygotowanie materiałów prezentacyjnych	50%

Tabela 3: Podział zadań i procentowy wkład członków zespołu

2 Wprowadzenie i motywacja

2.1 Kontekst problemu

Aplikacje webowe są stale narażone na automatyczny ruch generowany przez boty: masowe skanowanie zasobów, testowanie endpointów, tworzenie sztucznego ruchu lub działania fraudowe. Klasyczne podejścia ochronne oparte wyłącznie o reguły sieciowe i statyczne sygnatury przestają być wystarczające, ponieważ nowoczesne boty potrafią imitować część zachowań użytkownika.

2.2 Dlaczego behawioralna biometria myszy

Trajektoria kursora, rytm kliknięć, dynamika scrollowania i przejścia focus/blur niosą bogatą informację o sposobie interakcji człowieka z interfejsem. W odróżnieniu od pojedynczego eventu, analiza krótkiego okna zachowania:

- lepiej oddaje *styl* interakcji,
- utrudnia prostym botom przejście niezauważonym,
- daje możliwość szybkiej reakcji ochronnej jeszcze w trakcie sesji.

2.3 Cel projektu

Celem MVP było zbudowanie kompletnego pipeline'u *end-to-end*:

- frontend zbiera telemetrykę zachowania i fingerprint środowiska,
- backend zapisuje sesje i surowe eventy,
- moduł cech buduje reprezentację okien zachowania,
- model klasyfikuje `human` vs `bot`,
- silnik polityk zwraca akcję: `allow/observe/throttle/challenge/block`.

2.4 Zakres i świadome uproszczenia MVP

Wersja demonstracyjna została zaprojektowana profesjonalnie, ale celowo uproszczona:

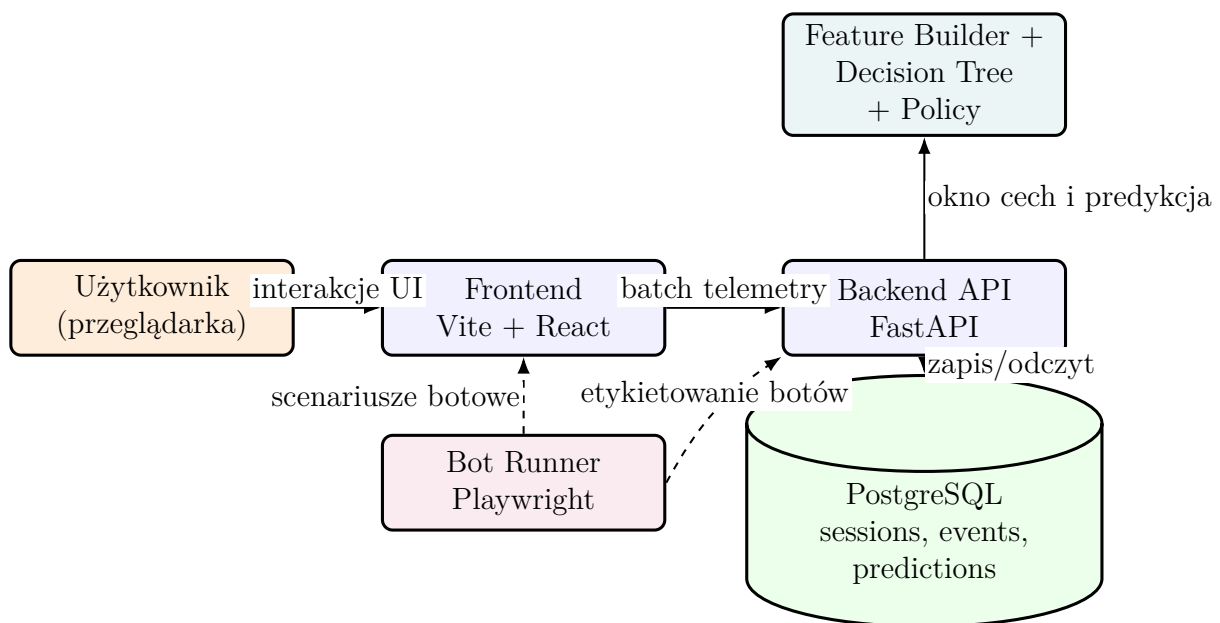
- inferencja modelu odbywa się w tym samym backendzie FastAPI (bez osobnego model service),
- predykcja działa na poziomie sesji i krótkich okien eventów,
- brak integracji z zewnętrznym WAF i blokadą IP (reakcja sesyjna),
- nacisk na czytelność kodu, odtwarzalność wyników i łatwą prezentację lokalną.

3 Architektura systemu

3.1 Widok komponentów

System składa się z czterech głównych części uruchamianych przez Docker Compose:

- **Frontend** (Vite + React + TypeScript) – zbiera telemetry użytkownika.
- **Backend** (FastAPI) – kolekcja eventów, predykcja, etykietowanie, API.
- **PostgreSQL** – trwale składowanie sesji, eventów, predykcji i akcji.
- **Bot Runner** (Playwright) – generowanie powtarzalnych scenariuszy botowych.

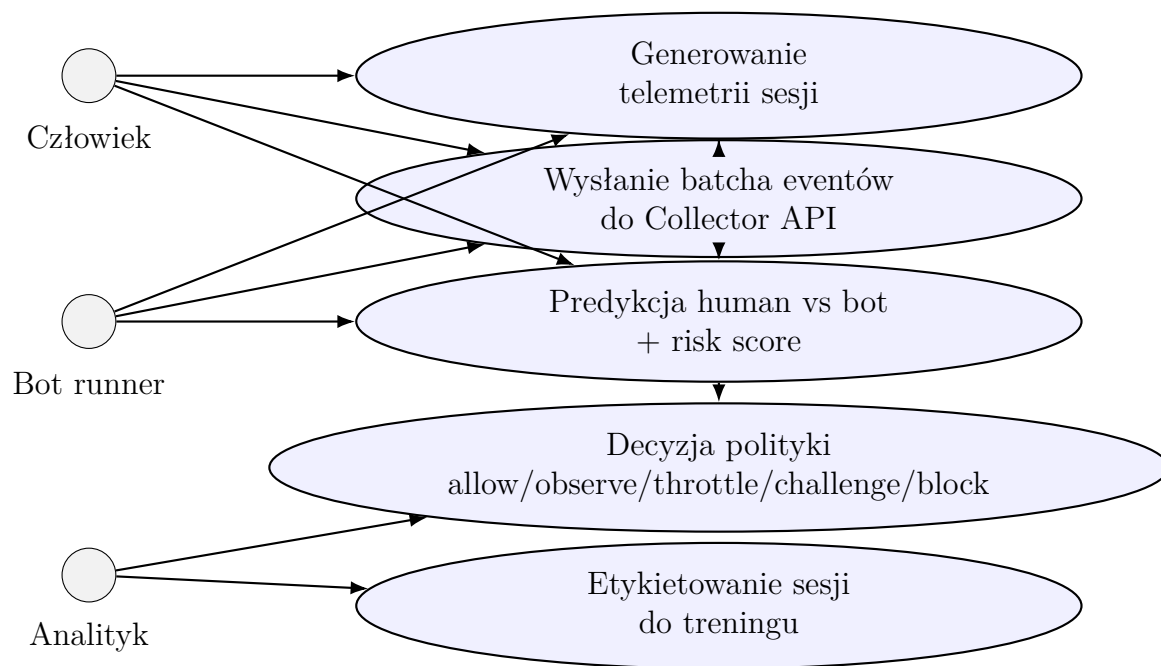


Rysunek 1: Architektura logiczna MVP

3.2 Przepływ danych end-to-end

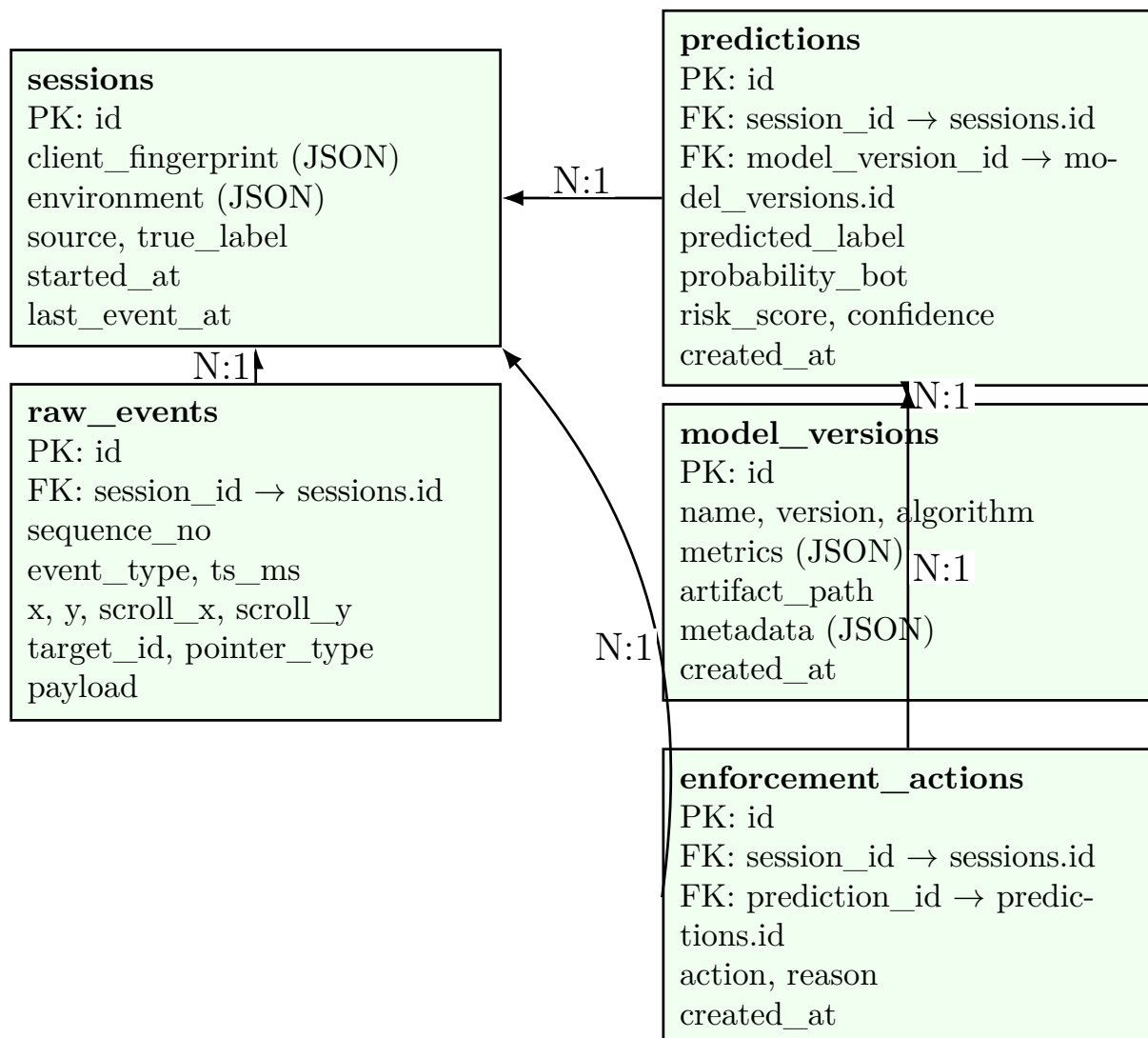
1. Frontend inicjuje sesję przez `POST /api/v1/sessions`.
2. Kolektor telemetry zbiera eventy i wysyła je batchami do `/api/v1/events/batch`.
3. Backend zapisuje surowe dane do tabel `sessions` i `raw_events`.
4. Dla żądania predykcji backend buduje cechy z okna eventów.
5. Model zwraca `predicted_label`, prawdopodobieństwo bota i `risk_score`.
6. Policy engine mapuje `risk_score` na akcję enforcement.
7. Wynik i akcja są zapisywane (`predictions`, `enforcement_actions`).

3.3 Diagram przypadków użycia



Rysunek 2: Główne przypadki użycia systemu

3.4 Model danych (ERD)



Rysunek 3: Relacyjny model danych PostgreSQL

4 Dokumentacja techniczna codebase

4.1 Struktura repozytorium

```
Projekt_ADAC/  
  backend/  
    app/  
      api/routes/ # endpointy FastAPI  
      core/ # konfiguracja, DB, logging  
      models/ # ORM + schematy request/response  
      repositories/ # dostep do danych  
      services/ # feature engineering, policy, telemetry, trening danych  
      ml/ # trening i ewaluacja modelu  
      scripts/ # seed i eksport replay  
      tests/ # testy jednostkowe
```



```
frontend/  
  src/  
    api/ # klient API  
    components/ # testbed UI  
    telemetry/ # collector + fingerprint + typy  
  bot_runner/  
    main.py # uruchamianie scenariuszy Playwright  
    scenarios.py # linear, human_like, adaptive, manual  
  docker-compose.yml  
  README.md  
  raport/ # dokumentacja LaTeX
```

4.2 Frontend testbed (Vite + React + TypeScript)

4.2.1 Rola frontendu

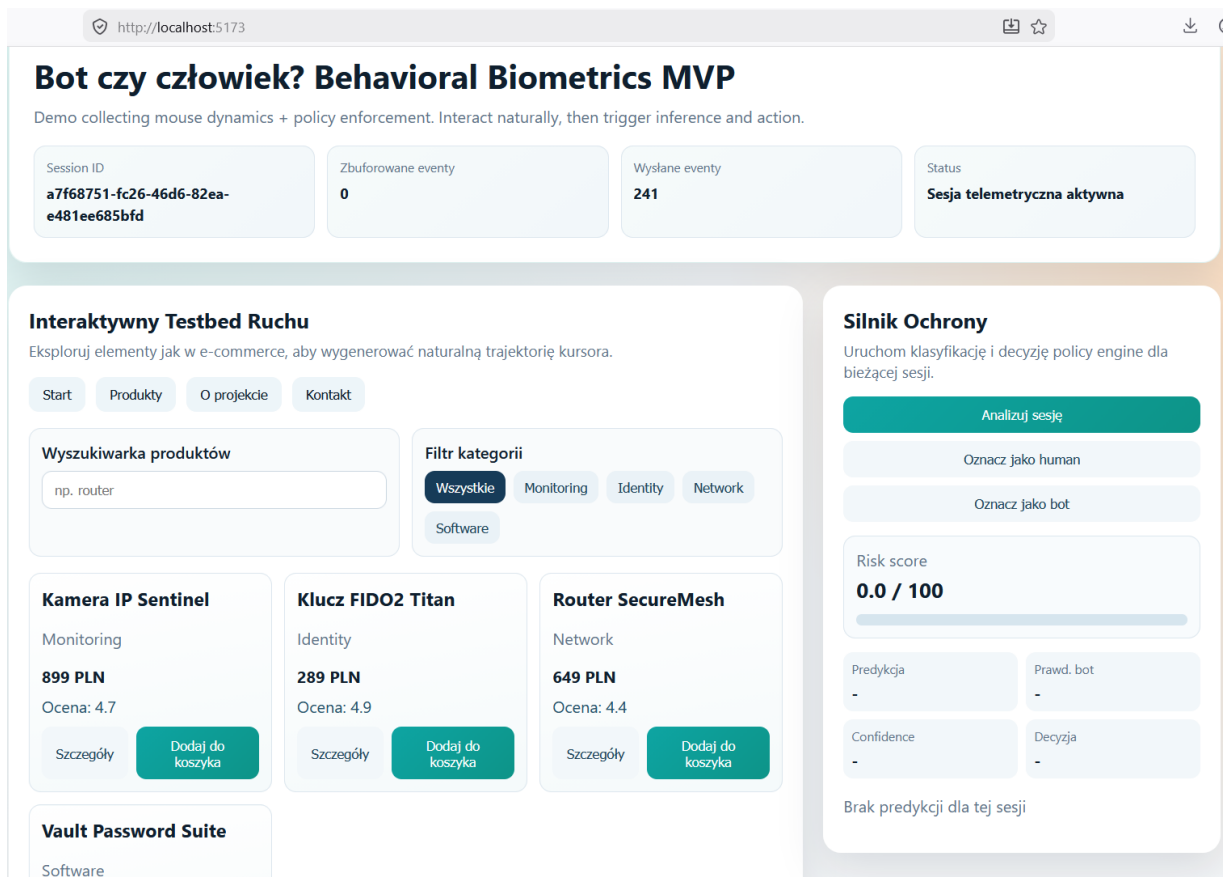
Frontend pełni podwójną funkcję:

- **generatora naturalnych interakcji użytkownika** (interfejs testowy),
- **sensora telemetrycznego** wysyłającego zdarzenia do backendu.

4.2.2 Wygląd i funkcje interfejsu

Widok frontendu pokazano na Rysunku 4. Układ ma trzy strefy:

- **Panel sesji telemetrycznej** (gora): `session id`, liczniki eventów (zbuforowane/wysłane) i status collector pipeline.
- **Interaktywny testbed ruchu** (lewa kolumna): menu, wyszukiwarka, filtry kategorii, karty produktowe i przyciski CTA. To celowo przypomina realistyczny scenariusz e-commerce.
- **Silnik ochrony** (prawa kolumna): uruchamianie predykcji, ręczne etykietowanie sesji (human/bot) i podgląd wyniku: `risk score`, `predykcja`, `confidence`, `decyzja`.



Rysunek 4: Frontend testbed na localhost: panel telemetryczny, strefa interakcji i panel policy engine

4.2.3 Zbierane eventy i batching

Kolektor (`frontend/src/telemetry/collector.ts`) rejestruje:

- `mousemove`,
- `click`,
- `scroll`,
- `focus`,
- `blur`,
- `viewport_enter/viewport_leave` (przez `visibilitychange`).

Wysyłka odbywa się partiami:

- `batchSize = 35`,
- okresowe `flush` co 2000 ms,
- `retry`: eventy wracają do bufora przy błędzie API.

4.2.4 Fingerprint klienta

Zbierane cechy srodowiskowe obejmują m.in.:

- `user_agent`, `language`, `timezone`,
- `screen_width/height`, `viewport_width/height`,
- `platform`, `pointer_type`,
- `webdriver`, `headless_hint`,
- `hardware_concurrency`, `device_memory`.

4.3 Pozyskanie danych treningowych (human vs bot)

4.3.1 Jak zbierano dane klasy human

Dane `human` były zbierane manualnie przez interakcje operatora z frontendem (Rysunek 4): ruch myszy po menu, filtrach i kartach, kliknięcia, scrollowanie i wpisywanie fraz w wyszukiwarce. Po zakończonej próbie sesje oznaczano przyciskiem `Oznacz jako human`.

4.3.2 Jak zbierano dane klas botowych

Sesje botowe generowano automatycznie przez `bot_runner`:

```
docker compose exec bot_runner python main.py --scenario linear --sessions 25 --record-video
docker compose exec bot_runner python main.py --scenario human_like --sessions 26 --record-video
docker compose exec bot_runner python main.py --scenario adaptive --sessions 50 --record-video
```

4.3.3 Udział scenariuszy botowych

Na poziomie sesji botowych przyjęto:

- `linear`: 25 sesji (24.75% klasy bot),
- `human_like`: 26 sesji (25.74% klasy bot),
- `adaptive`: 50 sesji (49.50% klasy bot).

Łącznie: 101 sesji botowych.

4.3.4 Charakterystyka botów i zamysł projektowy

- `linear`: ruch prosty i rytmiczny, niski realizm. Cel: łatwy baseline wykrywania prostych automatyzacji.
- `human_like`: krzywe Beziera, jitter i losowe opóźnienia. Cel: etap pośredni pomiędzy botem prostym i trudniejszym.

- **adaptive**: scenariusz kalibrowany profilem zachowań człowieka (`/api/v1/sessions/human-behavior`) z dynamicznym tempem ruchu, pauzami, korektami toru i bardziej realistycznym typingiem. Cel: utrudnić klasyfikację i przetestować odporność modelu na boty zbliżone dystrybucyjnie do klasy `human`.

4.3.5 Scenariusze eksperymentalne

W środowisku testowym przygotowano pięć scenariuszy eksperymentalnych:

1. **Human baseline**: realny użytkownik ręcznie korzysta z frontendu, wykonując ruchy kursorem, kliknięcia, scrollowanie i wpisywanie fraz.
2. **Bot linear**: automatyzacja przechodzi pomiędzy punktami po prostych trajektoriach i z relatywnie regularnym tempem.
3. **Bot human-like**: automatyzacja dodaje losowe opóźnienia, jitter oraz krzywe Béziera, aby częściowo imitować naturalne przejścia kursora.
4. **Bot adaptive**: automatyzacja korzysta z profilu zachowań człowieka pobieranego z backendu i próbuje dopasować rozkłady tempa, pauz i korekt ruchu.
5. **Bot replay**: odtworzenie wcześniej wyeksportowanej trajektorii sesji z pliku JSON. Ten wariant służy głównie do testów powtarzalności.

4.4 Generacja i przygotowanie danych

4.4.1 Opis zebranych danych

Końcowy zbiór na poziomie sesji obejmuje 202 oznaczone sesje:

- 101 sesji klasy `human`,
- 101 sesji klasy `bot`,
- w klasie `bot`: 25 sesji `linear`, 26 sesji `human_like` oraz 50 sesji `adaptive`.

Po zastosowaniu okien `window_size=120` i `stride=80` uzyskano 644 próbki okien zachowania wykorzystane w ewaluacji aktywnego modelu. Każda próbka reprezentuje fragment sesji, a nie pojedyncze zdarzenie.

4.4.2 Przygotowanie danych

Przed treningiem dane są przekształcane w następujący sposób:

- eventy są sortowane po `ts_ms`,
- okna zbyt krótkie są uzupełniane zerowymi wartościami cech, jeśli dana cecha nie może zostać policzona,
- różnice czasu są zabezpieczane przez `max(1, dt_ms)`, aby uniknąć dzielenia przez zero,
- wszystkie cechy są rzutowane na wartości liczbowe typu `float`,

- cechy środowiskowe są liczone, ale w finalnym drzewie wyłączone z modelu.

Nie stosowano normalizacji typu **StandardScaler**, ponieważ finalny model to drzewo decyzyjne. Drzewa są niewrażliwe na monotoniczne skalowanie cech, więc skalowanie nie poprawiałoby interpretowalności progów decyzyjnych.

4.4.3 Analiza jakości danych

Najważniejsze obserwacje dotyczące jakości danych:

- zbiór jest zbalansowany na poziomie sesji (101 **human** i 101 **bot**), co ogranicza problem dominacji jednej klasy,
- dane botowe są kontrolowane i powtarzalne, dzięki czemu dobrze nadają się do demonstracji pipeline'u,
- dane human pochodzą z lokalnego testbedu i ograniczonej liczby operatorów, co jest wystarczające dla MVP, ale ogranicza generalizację,
- cechy środowiskowe mogłyby sztucznie ułatwiać klasyfikację, dlatego zostały wyłączone z finalnego modelu,
- dominacja cechy **acc_max** wskazuje, że najtrudniejszym elementem do realistycznego zasymulowania jest nagła, impulsowa dynamika ruchu człowieka.

4.5 Backend Collector API (FastAPI)

4.5.1 Warstwy backendu

- **API routes** – walidacja wejścia/wyjścia (Pydantic) i kontrakty REST.
- **Services** – logika domenowa (telemetry, cechy, polityki, model).
- **Repositories** – operacje bazodanowe (SQLAlchemy ORM).
- **ML module** – trening i ewaluacja modelu.

4.5.2 Endpointy REST

Endpoint	Metoda	Opis
/api/v1/health	GET	Healthcheck aplikacji.
/api/v1/sessions	POST	Utworzenie nowej sesji i zapis fingerprintu.
/api/v1/events/batch	POST	Przyjęcie batcha eventów dla sesji.
/api/v1/sessions/{session_id}/label	POST	Etykietowanie sesji jako human lub bot .
/api/v1/predict/{session_id}	POST	Predykcja dla sesji + decyzja policy engine.

Endpoint	Metoda	Opis
/api/v1/predictions/{session_id}	GET	Pobranie ostatniej predykcji sesji.
/api/v1/sessions/{session_id}	GET	Status sesji i metadane.
/api/v1/sessions/human-behavior-profile	GET	Profil statystyczny zachowan czlowieka do adaptacji bota.

4.6 Storage: PostgreSQL

Model danych sklada sie z pieciu tabel:

- **sessions** – glowna encja sesji,
- **raw_events** – surowy strumien interakcji,
- **predictions** – wyniki modelu,
- **model_versions** – rejestr wersji modelu i metryk,
- **enforcement_actions** – decyzje polityki ochronnej.

Relacje opisuje diagram ERD w Sekcji 2.

4.7 Słownik zmiennych bazodanowych

Tabela 5 pokazuje wszystkie zmienne przechowywane w bazie PostgreSQL. Kolumna *czy wchodzi do modelu* oznacza, czy dana zmienna jest wykorzystywana w pipeline cech (bezposrednio lub przez transformacje).

Tabela 5: Słownik zmiennych bazodanowych: znaczenie i udział w modelu

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
sessions.id	identyfikator sesji (UUID).	NIE
sessions.source	zrodlo sesji (np. frontend / bot_runner).	NIE
sessions.true_label	etykieta referencyjna human/bot nadawana recznie lub automatycznie.	NIE
sessions.status	status procesu sesji (np. collecting , predicted).	NIE
sessions.client_fingerprint	surowy fingerprint klienta (JSON).	NIE
sessions.environment	surowe parametry srodowiskowe (JSON), zrodlo cech srodowiskowych.	TAK
sessions.created_at	czas utworzenia rekordu sesji.	NIE
sessions.updated_at	czas ostatniej aktualizacji sesji.	NIE
sessions.environment.user_agent	identyfikator przeglądarki/OS; uzywany do flag pomocniczych.	TAK
sessions.environment.language	jezyk przeglądarki.	NIE

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
sessions.environment.timezone	strefa czasowa klienta.	NIE
sessions.environment.screen_width	szerokosc ekranu klienta; skladnik screen_area .	TAK
sessions.environment.screen_height	wysokosc ekranu klienta; skladnik screen_area .	TAK
sessions.environment.viewport_width	szerokosc viewportu; skladnik viewport_area .	TAK
sessions.environment.viewport_height	wysokosc viewportu; skladnik viewport_area .	TAK
sessions.environment.webdriver	flaga automatyzacji przegladarki.	TAK
sessions.environment.headless_hint	sygnal trybu headless.	TAK
sessions.environment.hardware_concurrency	liczba logicznych rdzeni CPU raportowana przez przegladarke.	TAK
sessions.environment.device_memory	pamiec urzadzenia raportowana przez przegladarke.	TAK
raw_events.id	techniczny identyfikator eventu.	NIE
raw_events.session_id	klucz obcy do sesji.	NIE
raw_events.sequence_no	numer sekwencyjny eventu w batchu.	NIE
raw_events.event_type	typ eventu (mousemove , click , scroll , ...).	TAK
raw_events.ts_ms	timestamp eventu w milisekundach, baza do dynamiki.	TAK
raw_events.x	pozycja X kursora.	TAK
raw_events.y	pozycja Y kursora.	TAK
raw_events.scroll_x	przesuniecie poziome scrolla.	NIE
raw_events.scroll_y	przesuniecie pionowe scrolla, baza scroll_velocity .	TAK
raw_events.target_id	identyfikator celu eventu; baza hover/target_revisit .	TAK
raw_events.target_tag	tag HTML celu interakcji.	NIE
raw_events.target_class	klasy CSS celu interakcji.	NIE
raw_events.pointer_type	typ urzadzenia wskazujacego.	NIE
raw_events.in_viewport	flaga widoczności eventu w viewport.	NIE
raw_events.payload	dodatkowy payload zdarzenia (JSON).	NIE
raw_events.created_at	czas zapisu eventu.	NIE
predictions.id	identyfikator predykcji.	NIE
predictions.session_id	sesja, dla ktorej wykonano predykcje.	NIE
predictions.model_version_id	wersja modelu uzyta do inferencji.	NIE
predictions.predicted_label	wynik klasyfikacji modelu (human/bot).	NIE
predictions.probability_bot	prawdopodobienstwo klasy bot .	NIE
predictions.confidence	pewnosc predykcji (maksimum prawdopodobienstwa).	NIE
predictions.risk_score	przeskalowany wynik ryzyka 0–100.	NIE
predictions.feature_snapshot	snapshot wektora cech wykorzystanych przy predykcji.	NIE
predictions.created_at	czas utworzenia predykcji.	NIE
model_versions.id	identyfikator wersji modelu.	NIE
model_versions.name	nazwa modelu.	NIE
model_versions.version	oznaczenie wersji (np. timestamp).	NIE
model_versions.algorithm	algorytm (Decision Tree).	NIE
model_versions.feature_set_version	wersja zestawu cech.	NIE
model_versions.metrics	metryki treningowe/ewaluacyjne (JSON).	NIE
model_versions.artifact_path	sciezka do artefaktu modelu.	NIE
model_versions.model_metadata	metadane modelu (JSON).	NIE

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
model_versions.is_active	flaga aktywnej wersji modelu.	NIE
model_versions.created_at	czas rejestracji wersji modelu.	NIE
enforcement_actions.id	identyfikator decyzji polityki.	NIE
enforcement_actions.session_id	sesja, ktorej dotyczy decyzja.	NIE
enforcement_actions.prediction_id	predykcja powiazana z decyzja.	NIE
enforcement_actions.action	decyzja (allow/observe/throttle/challenge/block).	NIE
enforcement_actions.reason	uzasadnienie decyzji polityki.	NIE
enforcement_actions.blocked_until	koniec blokady czasowej (jesli dotyczy).	NIE
enforcement_actions.action_metadata	dodatkowe metadane decyzji (JSON).	NIE
enforcement_actions.created_at	czas zapisu decyzji.	NIE

4.8 Feature engineering

Pipeline cech (`backend/app/services/feature_engineering.py`) liczy cechy z okna eventow posortowanych po czasie (`ts_ms`), a nie z pojedynczego eventu.

4.8.1 Przykladowe cechy dynamiczne

- trajektoria i geometria: `path_length`, `displacement`, `path_efficiency`,
- dynamika ruchu: `speed_mean/std/max`, `acc_mean/std/max`, `jerk_mean/std/max`,
- mikrozachowania: `micro_pause_count`, `correction_rhythm`, `overshoot_count`,
- klikniecia: `click_interval_mean/std`, `hover_before_click_mean`, `target_revisit_count`,
- kontekst sekwencji: `event_type_entropy`, `unique_event_types`,
- scroll i focus: `scroll_velocity_mean/std`, `focus_blur_transitions`.

4.8.2 Cechy srodowiskowe

Pipeline zawiera takze cechy srodowiskowe, jednak w finalnym modelu drzewa sa one **domyslnie wykluczone z treningu** (`feature_set=behavioral`), aby ograniczyc ryzyko data leakage (np. trywialny podzial po `viewport_area`).

4.8.3 Slownik zmiennych cechowych

Tabela 6 zawiera komplet zmiennych cechowych budowanych w pipeline i informacje, czy dana zmienna wchodzi do finalnego modelu.

Tabela 6: Słownik zmiennych cechowych: nazwa, znaczenie i użycie w modelu finalnym

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
event_count	liczba wszystkich eventów w oknie czasowym.	TAK
window_duration_ms	czas trwania okna: $ts_last - ts_first$.	TAK
mousemove_count	liczba eventów <code>mousemove</code> .	TAK
click_count	liczba eventów <code>click</code> .	TAK
scroll_count	liczba eventów <code>scroll</code> .	TAK
focus_blur_transitions	suma liczby eventów <code>focus</code> i <code>blur</code> .	TAK
viewport_enter_count	liczba eventów <code>viewport_enter</code> .	TAK
viewport_leave_count	liczba eventów <code>viewport_leave</code> .	TAK
path_length	suma odległości euklidesowych pomiędzy kolejnymi punktami kursora.	TAK
displacement	odległość między pierwszym i ostatnim punktem kursora w oknie.	TAK
path_efficiency	współczynnik $displacement / \max(path_length, 1)$.	TAK
speed_mean	średnia prędkość kursora pomiędzy kolejnymi punktami ($distance/dt$).	TAK
speed_std	odchylenie standardowe prędkości kursora.	TAK
speed_max	maksymalna prędkość kursora w oknie.	TAK
acc_mean	średnie przyspieszenie: $\Delta(speed)/dt$.	TAK
acc_std	odchylenie standardowe przyspieszenia.	TAK
acc_max	maksymalne przyspieszenie w oknie.	TAK
jerk_mean	średni jerk: $\Delta(acc)/dt$.	TAK
jerk_std	odchylenie standardowe jerk.	TAK
jerk_max	maksymalny jerk w oknie.	TAK
curvature_mean	średnia krzywizna toru: $ \Delta(kat) / \max(distance, 1)$.	TAK
curvature_std	odchylenie standardowe krzywizny toru.	TAK
angle_change_mean	średnia zmiana kąta ruchu pomiędzy kolejnymi segmentami.	TAK
angle_change_std	odchylenie standardowe zmian kąta ruchu.	TAK
micro_pause_count	liczba segmentów z bardzo niską prędkością ($speed < 40$ px/s).	TAK
correction_rhythm	liczba mocnych korekt kierunku (prog zmiany kąta > 1.1 rad).	TAK
click_interval_mean	średni odstęp czasowy między kolejnymi kliknięciami.	TAK
click_interval_std	odchylenie standardowe odstępów między kliknięciami.	TAK
hover_before_click_mean	średni czas hover przed kliknięciem, liczony lokalnie przy celu kliknięcia.	TAK
dwell_time_mean	średni czas przebywania na segmentach ($1000/\max(speed, 1)$).	TAK
overshoot_count	liczba przypadków overshoot (rewersy kierunku przed kliknięciem).	TAK
target_revisit_count	liczba celów klikanych wielokrotnie w tej samej sesji okna.	TAK
scroll_velocity_mean	średnia prędkość scrolla: $ \Delta(scroll_y) /dt$.	TAK
scroll_velocity_std	odchylenie standardowe prędkości scrolla.	TAK
unique_event_types	liczba unikalnych typów eventów w oknie.	TAK

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
event_type_entropy	entropia Shannona rozkładu typów eventów w oknie.	TAK
webdriver_flag	flaga 0/1: czy <code>navigator.webdriver</code> jest prawdą.	NIE
headless_flag	flaga 0/1: wykrycie sygnału headless na podstawie <code>user-agent</code> .	NIE
viewport_area	pole viewportu: <code>viewport_width * viewport_height</code> .	NIE
screen_area	pole ekranu: <code>screen_width * screen_height</code> .	NIE
viewport_screen_ratio	iloraz <code>viewport_area / max(screen_area, 1)</code> .	NIE
hardware_concurrency	liczba logicznych rdzeni CPU raportowana przez przeglądarkę.	NIE
device_memory	pamięć urządzenia raportowana przez przeglądarkę.	NIE

4.9 Model i inferencja

4.9.1 Trening offline

Skrypt `python -m app.ml.train` realizuje:

- budowę `DataFrame` z okien (`window_size`, `stride`),
- podział `train/test` z grupowaniem po `session_id` (`GroupShuffleSplit`),
- trening `DecisionTreeClassifier` z `class_weight=balanced`,
- zapis artefaktów: model, metryki, ROC, confusion matrix, drzewo i reguły.

4.9.2 Predykcja online

Inferencja działa przez `ModelManager`:

- ładuje aktywny artefakt modelu (`active_model.joblib`),
- zwraca `predicted_label`, `probability_bot`, `confidence`, `risk_score`,
- przy braku modelu używa fallbacku heurystycznego.

4.10 Policy engine / enforcement

Mapowanie ryzyka (`backend/app/services/policy_engine.py`):

- `risk < 25`: allow,
- `25-50`: observe,
- `50-75`: throttle,
- `75-90`: challenge,
- `>= 90`: block.

4.11 DevOps i uruchomienie

Srodowisko dziala lokalnie przez Docker Compose:

- `frontend` – port 5173,
- `backend` – port 8000,
- `db` – port 5432,
- `bot_runner` – kontener narzedziowy do generacji sesji.

4.12 Jakosc i testy

- testy jednostkowe backendu (`backend/app/tests`),
- walidacja kontraktow API przez schematy Pydantic,
- separacja odpowiedzialnosci: API/Services/Repositories/ML.

5 Model finalny i raport wynikow

5.1 Cel i zakres analizy modelu

W tej sekcji dokumentujemy finalny model klasyfikacji `human` vs `bot`, z naciskiem na:

- jakie dane i cechy weszly do treningu,
- jak wykonano podzial na trening i test,
- jak przeprowadzono walidacje i ewaluacje,
- na ile wyniki odpowiadaja rzeczywistemu dzialaniu systemu.

5.2 Konfiguracja modelu finalnego

Finalna wersja modelu to drzewo decyzyjne trenowane na cechach behawioralnych:

- algorytm: `DecisionTreeClassifier`,
- parametry okna: `window_size = 120, stride = 80`,
- hiperparametry: `max_depth = 12, min_samples_leaf = 2`,
- `class_weight = balanced`,
- `feature_set = behavioral` (bez cech srodowiskowych).

5.3 Uzasadnienie wyboru modelu

W finalnej wersji projektu zastosowano `DecisionTreeClassifier`, ponieważ łączy trzy cechy ważne dla raportu końcowego:

- **interpretowalność**: decyzje można przedstawić jako jawne reguły i diagram drzewa,
- **brak konieczności normalizacji**: model może pracować bezpośrednio na cechach liczbowych o różnych skalach,
- **czytelność demonstracyjna**: łatwo pokazać, które progi cech prowadzą do klasyfikacji human albo bot.

W projekcie celowo skupiono się na jednym finalnym, audytowalnym modelu drzewa, zamiast utrzymywać kilka modeli równolegle. Jest to zgodne z charakterem MVP: priorytetem była możliwość zrozumienia decyzji modelu i pokazania jej na diagramie.

5.4 Jakie dane weszły do modelu

5.4.1 Poziom surowy

Wejściem są sesje zapisane w PostgreSQL (`sessions`, `raw_events`), z etykieta referencyjna `true_label` nadana przez operatora. Każde okno zawiera uporządkowany po czasie fragment zachowania, a nie pojedynczy event.

5.4.2 Poziom cech

Z okna eventów budowane są cechy opisujące:

- dynamikę ruchu kursora: `speed_*`, `acc_*`, `jerk_*`,
- geometrię toru: `path_length`, `path_efficiency`, `curvature_*`,
- mikrozachowania: `micro_pause_count`, `correction_rhythm`, `overshoot_count`,
- wzorzec kliknięć i scrolla: `click_interval_*`, `hover_before_click_mean`, `scroll_velocity_*`,
- entropie sekwencji: `event_type_entropy`, `unique_event_types`.

W finalnym modelu świadomie pominięto cechy środowiskowe (np. `viewport_area`), żeby ograniczyć ryzyko sztucznego zawyżania wyniku przez data leakage.

5.5 Dobór zmiennych: proces i uzasadnienie

5.5.1 Przestrzeń kandydacka i finalny wybór

Proces startował od pełnego słownika cech wyliczanego przez pipeline (`FEATURE_COLUMNS`) opisany w Tabeli 6. W praktyce były to:

- 43 cechy kandydackie łącznie,
- 7 cech środowiskowych,

- 36 cech behawioralnych.

Do finalnego modelu weszły cechy behawioralne (`feature_set=behavioral`), natomiast cechy środowiskowe zostały wyłączone z trenowania drzewa.

5.5.2 Kryteria doboru zmiennych

Selekcja była oparta o trzy kryteria:

- **odpornosc na leakage:** odciecie sygnałów typu `viewport_area` i `screen_area`, które mogą rozdzielać klasy na skroty, bez realnej analizy zachowania,
- **związek z mikrodynamiką ruchu:** preferencja dla cech opisujących kinetykę i geometrię toru (`acc_*`, `speed_*`, `curvature_*`),
- **interpretowalność:** drzewo z tak dobranym zbiorem cech daje reguły decyzyjne, które można odczytać i wyjaśnić.

5.5.3 Co oznacza ten wybór w praktyce

Model finalny klasyfikuje sesje głównie na podstawie *jak* porusza się kursor i jak wygląda rytm interakcji, a nie *na czym* sesja jest wykonywana. To zwiększa wiarygodność demonstracji, bo decyzja nie wynika z jednego artefaktu środowiskowego, tylko z profilu zachowania.

5.6 Podział train/test i walidacja

5.6.1 Strategia podziału

Podział został wykonany przez `GroupShuffleSplit` z `test_size=0.25` i grupowaniem po `session_id`. To kluczowe, bo:

- okna z tej samej sesji nie trafiają jednocześnie do train i test,
- test sprawdza uogólnienie na nieznane sesje, a nie tylko nieznane okna.

5.6.2 Co oznaczają liczby w praktyce

Z raportu metryk treningowych (Tabela 7 i Tabela 8) wynika, że zbiór testowy miał 136 okien ($TN+FP+FN+TP = 74+1+6+55$). W ewaluacji aktywnego modelu analizowano 644 okna i 202 sesje (Tabela 10).

5.6.3 Typ walidacji

W MVP użyto pojedynczego hold-out splitu grupowego (jedna próbka walidacyjna). To dobre rozwiązanie demonstracyjne, ale nie zastępuje pełnej walidacji krzyżowej np. `GroupKFold`.

5.7 Metryki ilosciowe

5.7.1 Metryki z treningu (hold-out, GroupShuffleSplit)

Metryka	Wartosc
Precision	0.9821
Recall	0.9016
F1-score	0.9402
ROC-AUC	0.9442
False Positive Rate (human)	0.0133

Tabela 7: Metryki modelu na zbiorze testowym podczas treningu

Element	Wartosc
TN	74
FP	1
FN	6
TP	55

Tabela 8: Confusion matrix (trening)

5.7.2 Metryki ewaluacyjne (aktywny model)

Metryka	Wartosc
Precision	0.9940
Recall	0.9651
F1-score	0.9794
ROC-AUC	0.9815
False Positive Rate (human)	0.0021

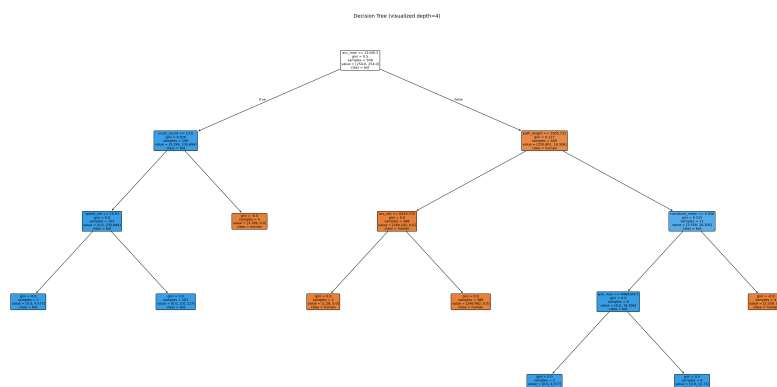
Tabela 9: Metryki z raportu ewaluacyjnego

Wskaznik	Wartosc
Liczba probek	644
Liczba sesji	202
Avg events to detect bot	122.2574
Median events to detect bot	120.0000
Detection delay count	101
Unseen bot recall count	0

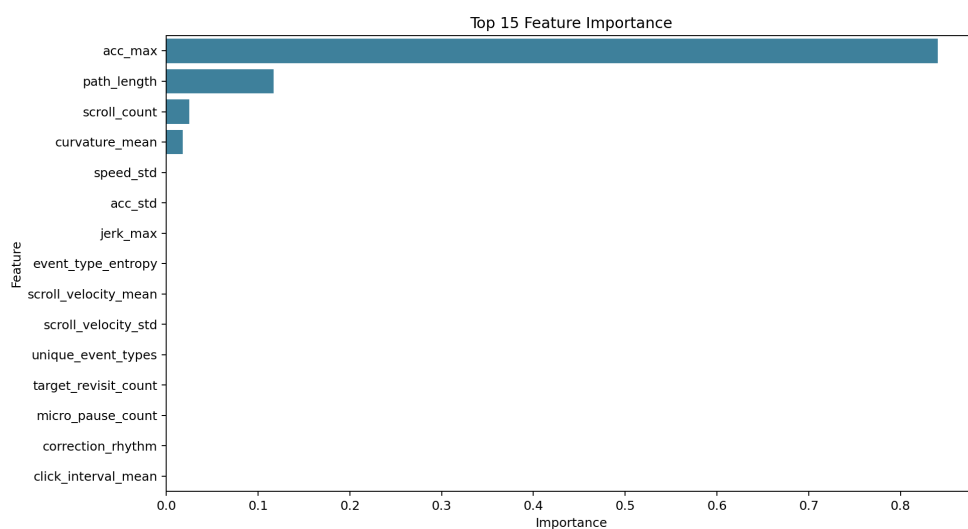
Tabela 10: Statystyki operacyjne ewaluacji

Wyniki z testu hold-out (Tabela 7) sa mocne (F1=0.9402, ROC-AUC=0.9442) przy niskim false positive rate dla klasy human (0.0133).

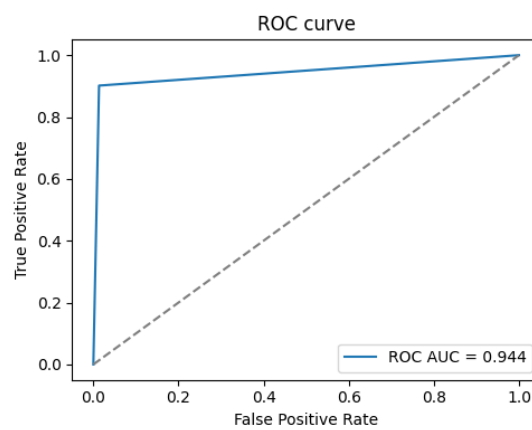
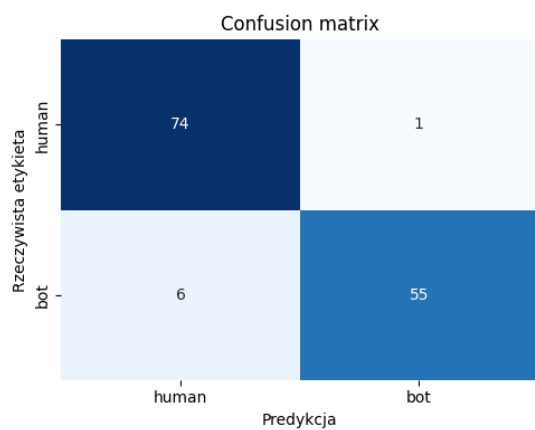
5.8 Wizualizacje modelu



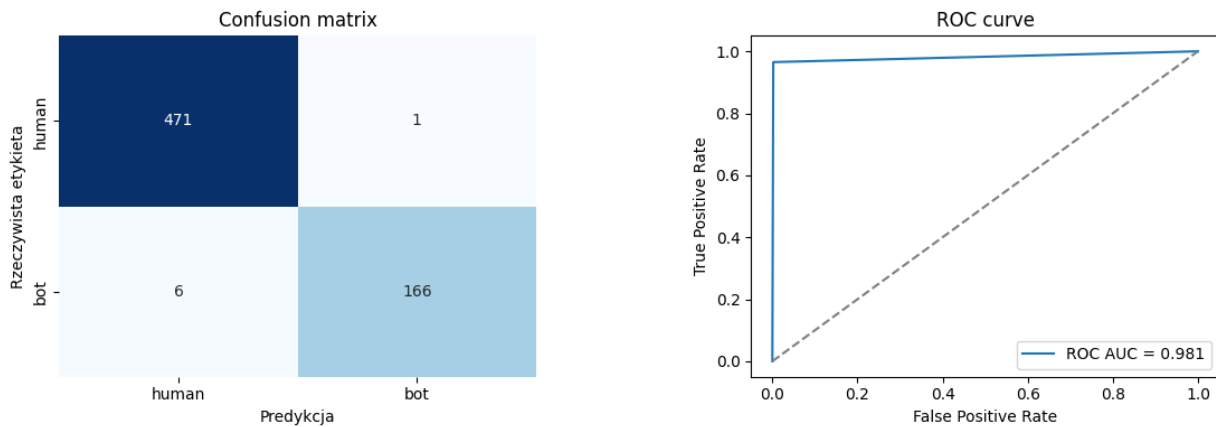
Rysunek 5: Diagram drzewa decyzyjnego (warstwy najwyższego poziomu)



Rysunek 6: Waznosc cech (feature importance) dla modelu finalnego



Rysunek 7: Confusion matrix oraz ROC dla zbioru testowego w treningu



Rysunek 8: Confusion matrix oraz ROC w ewaluacji aktywnego modelu

5.9 Interpretacja: co model rzeczywiscie robi

Rysunek 5 pokazuje, że model jest interpretowalny i można odczytać jawne warunki decyzji. Rysunek 6 potwierdza, że dominująca cecha to nadal `acc_max`, a kolejne sygnały (np. `path_length`, `event_type_entropy`) są uzupełniające.

5.9.1 Struktura decyzyjna drzewa: interpretacja warstw

Na podstawie Rysunku 5 i pełnych reguł można odczytać następującą logikę:

- **Wezeł główny (`acc_max`):** pierwszy podział oddziela sesje o niskich i wysokich pikach przyspieszenia kursora.
- **Galaz niskiego `acc_max`:** kolejne warunki `scroll_count` i `speed_std` prowadzi głównie do klasy `bot`. Intuicyjnie: ruch jest bardziej wygładzony i mniej impulsywny.
- **Galaz wysokiego `acc_max`:** podziały po `path_length` i `acc_std` częściej prowadzi do klasy `human`, bo odzwierciedlają bardziej zrywny i korekcyjny tor ruchu.
- **Wezły głębsze (`curvature_mean`, `jerk_max`)** doprecyzowują decyzje dla przypadków granicznych, gdzie sama dynamika pierwszego rzędu nie wystarcza.

5.9.2 Jak czytać diagram ważności cech

Rysunek 6 przedstawia ważności typu impurity-based (spadek nieczystości węzłów, Gini). Interpretacja:

- wysoka ważność `acc_max` oznacza, że ta cecha najczęściej daje największy przyrost informacji przy podziałach,
- niezerowa rola `path_length` i `event_type_entropy` pokazuje, że model korzysta także z geometrii toru i różnorodności sekwencji eventów,
- niska ważność wielu cech nie oznacza, że są globalnie niepotrzebne; oznacza, że w aktualnej próbie były rzadziej wybierane jako najlepszy podział.

5.9.3 Znaczenie operacyjne (co to oznacza dla detekcji)

Struktura drzewa jest spojna z celem systemu:

- sesje o zbyt gładkiej mikrodynamice są przesuwane w stronę **bot**,
- sesje z naturalnymi korektami i zrywami ruchu są przesuwane w stronę **human**,
- policy engine otrzymuje wynik oparty o zachowanie, a nie o pojedynczy artefakt środowiskowy.

5.10 Reguły klasyfikacji (fragment)

Model pozostaje interpretowalny – można odczytać reguły decyzyjne:

Listing 1: Przykładowy fragment reguł drzewa decyzyjnego

```
|--- acc_max <= 21440.2998
| |--- scroll_count <= 13.0000
| | |--- speed_std <= 19.9304 -> class: bot
|--- acc_max > 21440.2998
| |--- path_length <= 2905.7310
| | |--- acc_std > 6430.7249 -> class: human
```

Listing 1 jest bezpośrednim dopełnieniem Rysunku 5: po podziale na `acc_max` drzewo doprecyzowuje decyzje przez `scroll_count`, `speed_std`, `path_length` i `acc_std`. Taka sekwencja warunków jest czytelna biznesowo i łatwa do audytu.

5.11 Czy ewaluacja jest spojna z rzeczywistoscia?

5.11.1 Co jest spójne

- Hold-out test (Rysunek 7) i metryki z Tabeli 7 pokazują, że model dobrze odróżnia klasy na nieznanymi sesjach testowych.
- Metryka `detection delay` (Tabela 10) daje praktyczny sygnał, po ilu eventach system zwykle wykrywa bota.

5.11.2 Gdzie jest ryzyko przeszacowania

- Ewaluacja aktywnego modelu (Tabela 9, Rysunek 8) liczona jest na całym zbiorze oznaczonych okien, więc zawiera również dane wykorzystane do treningu.
- To oznacza, że wyniki ewaluacyjne są bardziej optymistyczne niż hold-out i nie powinny być jedynym kryterium oceny generalizacji.

5.11.3 Najważniejsze ograniczenie obecnego raportu

W Tabeli 10 `unseen_bot_recall` ma `count=0`, czyli w aktualnym zestawie nie ma osobnej, niewidzianej rodziny botów do twardego testu OOD. Dlatego aktualny raport dobrze opisuje zachowanie modelu *w ramach obecnej dystrybucji*, ale nie zamyka tematu odporności na nowe klasy botów.

5.12 Wpływ jakości danych na wyniki

Jakość danych ma bezpośredni wpływ na interpretację metryk:

- **Zbalansowane klasy:** równa liczba sesji **human** i **bot** ogranicza ryzyko modelu trywialnie preferującego klasę większościową.
- **Dane syntetyczne botów:** boty są kontrolowane, co ułatwia demonstrację, ale może zawyżać wynik względem realnych botów produkcyjnych.
- **Dane human z testbedu:** sesje człowieka są realistyczne w ramach lokalnego UI, ale nie obejmują szerokiej populacji urządzeń i użytkowników.
- **Dominacja `acc_max`:** wysoka ważność tej cechy oznacza, że obecne boty nadal różnią się od człowieka przede wszystkim dynamiką nagłych zrywów kursora. To informuje, gdzie należy rozwijać kolejne scenariusze botowe.
- **Wyłączenie cech środowiskowych:** ogranicza leakage, ale jednocześnie zmusza model do korzystania z sygnałów behawioralnych, co lepiej odpowiada celowi projektu.

5.13 Ograniczenia trafności wyników

Poniżej wskazano główne ryzyka interpretacyjne wyników:

- **Internal validity:** mimo wykluczenia części cech środowiskowych z podziałów drzewa, pozostaje ryzyko, że model wykorzystuje sygnały specyficzne dla aktualnego UI i sposobu testowania.
- **Construct validity:** etykieta **human** opiera się na ręcznym oznaczaniu sesji, co jest praktyczne, ale może zawierać szum (np. sesja człowieka z nienaturalnym rytmem).
- **External validity:** dane pochodzą głównie z lokalnego środowiska (localhost), ograniczonej liczby urządzeń/przeglądarek i kontrolowanych scenariuszy.
- **Conclusion validity:** zastosowano pojedynczy hold-out split grupowy, bez wielokrotnej walidacji (GroupKFold) i bez przedziałów ufności metryk.
- **OOD validity:** brak niezależnej rodziny botów w metryce `unseen_bot_recall` (`count=0`) utrudnia twardą ocenę odporności na nowe techniki automatyzacji.

Wniosek praktyczny: metryki są mocne dla aktualnej dystrybucji danych, ale produkcyjna pewność wymaga dalszego rozszerzania zbioru i walidacji wielokrotnej.

5.14 Wniosek końcowy

Model finalny jest mocny i interpretowalny na aktualnym zbiorze (patrz Tabela 7 i Rysunek 5), jednak dla pełnej zgodności z warunkami produkcyjnymi potrzebuje kolejnego kroku: oddzielnego testu na niewidzianych scenariuszach botowych oraz walidacji wielokrotnej na poziomie sesji.

6 Podsumowanie projektu

6.1 Co zostało osiągnięte

W ramach MVP dostarczono działający system *end-to-end*, który:

- zbiera zachowania użytkownika w aplikacji webowej,
- buduje cechy behawioralne z krótkich okien czasowych,
- klasyfikuje sesje **human** vs **bot**,
- generuje decyzję enforcement na podstawie **risk_score**,
- umożliwia trenowanie i ewaluację modelu na danych lokalnych.

6.2 Wnioski praktyczne

- Podejście behawioralne daje dobrą wykrywalność nawet przy botach imitujących człowieka.
- Interpretowalne drzewo decyzyjne ułatwia prezentację i audyt decyzji modelu.
- Największy wpływ nadal mają cechy dynamiki ruchu (zwłaszcza **acc_max**), więc doskonalenie botów powinno iść w stronę realistycznego modelowania nagłych zmian tempa.

6.3 Rekomendowane kolejne kroki

1. Rozszerzyć zbiór o nowe rodziny botów i sesje *unseen*.
2. Dodać walidację czasową (time-based split) oraz monitoring driftu cech.
3. Uzupełnić policy engine o reputację źródeł i denylistę.
4. Rozważyć model sekwencyjny (np. Transformer/RNN) jako warstwę drugą.
5. Dodać dashboard operacyjny do przeglądu sesji i interpretacji predykcji.

7 Materiały dodatkowe

7.1 Repozytorium projektu

Kod źródłowy projektu jest dostępny w repozytorium:

https://github.com/PiotrKosiara/Projekt_ADAC

Repozytorium zawiera:

- kod frontendu testbedowego,
- backend FastAPI wraz z modelem danych i endpointami,

- pipeline feature engineering i treningu modelu,
- bot runner Playwright,
- konfigurację Docker Compose,
- raport LaTeX oraz materiały prezentacyjne.

7.2 Struktura datasetu

Dane projektowe są przechowywane przede wszystkim w PostgreSQL. Logicznie dataset można podzielić na cztery warstwy:

1. **Warstwa sesji:** tabela `sessions`, czyli identyfikator sesji, etykieta referencyjna, źródło danych i fingerprint środowiska.
2. **Warstwa surowych eventów:** tabela `raw_events`, czyli chronologiczny zapis `mousemove`, `click`, `scroll`, `focus`, `blur` i zdarzeń viewportu.
3. **Warstwa cech:** okna zachowania generowane dynamicznie przez backend, z cechami takimi jak `acc_max`, `speed_std`, `path_length`, `event_type_entropy`.
4. **Warstwa wyników:** tabele `predictions`, `model_versions` i `enforcement_actions`.

7.3 Rozmiar i skład danych końcowych

Końcowy zestaw danych wykorzystany w raporcie:

- 202 oznaczone sesje,
- 101 sesji human,
- 101 sesji bot,
- 644 próbki okien zachowania w ewaluacji aktywnego modelu,
- scenariusze botowe: `linear` – 25 sesji, `human_like` – 26 sesji, `adaptive` – 50 sesji.

7.4 Odtwarzalność eksperymentu

Podstawowy eksperyment można odtworzyć lokalnie przez:

```
docker compose up -d --build
docker compose exec bot_runner python main.py --scenario linear --sessions 25 --record-video
docker compose exec bot_runner python main.py --scenario human_like --sessions 26 --record-video
docker compose exec bot_runner python main.py --scenario adaptive --sessions 50 --record-video
docker compose exec backend python -m app.ml.train --window-size 120 --stride 80 --tree-max-depth 12 --tree-min-samples-leaf 2
```

Dane human wymagają ręcznych interakcji z frontendem i etykietowania sesji jako human. Ten element jest celowo manualny, ponieważ stanowi referencyjne zachowanie człowieka dla scenariuszy botowych.